Task1:

String Similarity

Aim:-To calculate the sum of similarities between a given string *S*and each of its suffixes. Similarity is defined as the length of the longest common prefix between two strings.

Algorithm:-

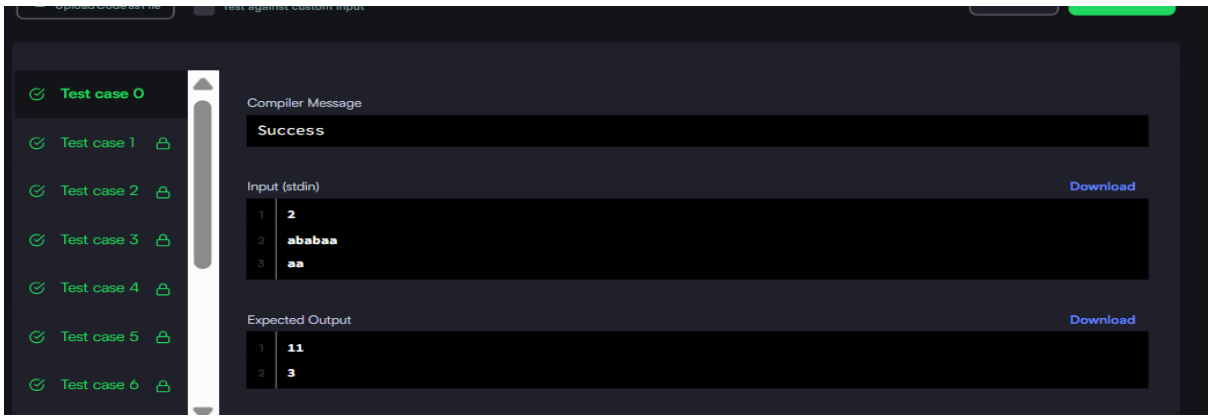
1. We use the Z-algorithm.
2. Build a Z-array where Z[i] represents the length of the longest substring starting at position i that matches the prefix of the string.
3. Initialize sum = n (the string itself is always fully similar to its own prefix).
4. For each suffix starting at index i, add Z[i] to the sum.
5. Return the total sum. This runs in O(n) time, which is optimal compared to a naive $O(n^2)$ approach.

Program:-

```
import java.io.*;

class Result {

   public static long stringSimilarity(String s) {

      int n = s.length();

      int[] z = new int[n];

      int l = 0, r = 0;

      long sum = n;

      for (int i = 1; i < n; i++) {

         if (i <= r) z[i] = Math.min(r - i + 1, z[i - l]);

         while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;

         if (i + z[i] - 1 > r) {

            l = i;

            r = i + z[i] - 1;
```

```
            }
            sum += z[i];
        }
        return sum;
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
        InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        for (int i = 0; i < t; i++) {
            String s = br.readLine().trim();
            System.out.println(Result.stringSimilarity(s));
        }
    }
}
```

Output:-



Result:-Thus,To calculate the sum of similarities between a given string *S*and each of its suffixes. Similarity is defined as the length of the longest common prefix between two strings is are successfully executed.

Task2:

Repeated Substring Pattern

Aim:- To determine whether a given string *s*can be constructed by repeating a substring multiple times.

Algorithm:-

1. If a string can be formed by repeating a substring, then the string will appear inside the concatenation of itself twice.
2. Example: "abab" → "abababab". Removing first and last char → "bababa". It contains "abab".
3. Concatenate the string with itself: s + s.
4. Remove the first and last character.
5. Check if the original string s exists inside this new string.
6. If yes → return true. Otherwise → return false.

Program:-

```
class Solution {
   public boolean repeatedSubstringPattern(String s) {
      String doubled = s + s;
      return doubled.substring(1, doubled.length() - 1).contains(s);
   }
}
```

Output:-

Result:-Thus,To determine whether a given string *s*can be constructed by repeating a substring multiple times is are successfully executed.

Task3:

Two Strings

Aim:- To determine if two given strings share a common substring. Since a substring can be as small as one character, the problem reduces to checking if the two strings share at least one common character.

Algorithm:-

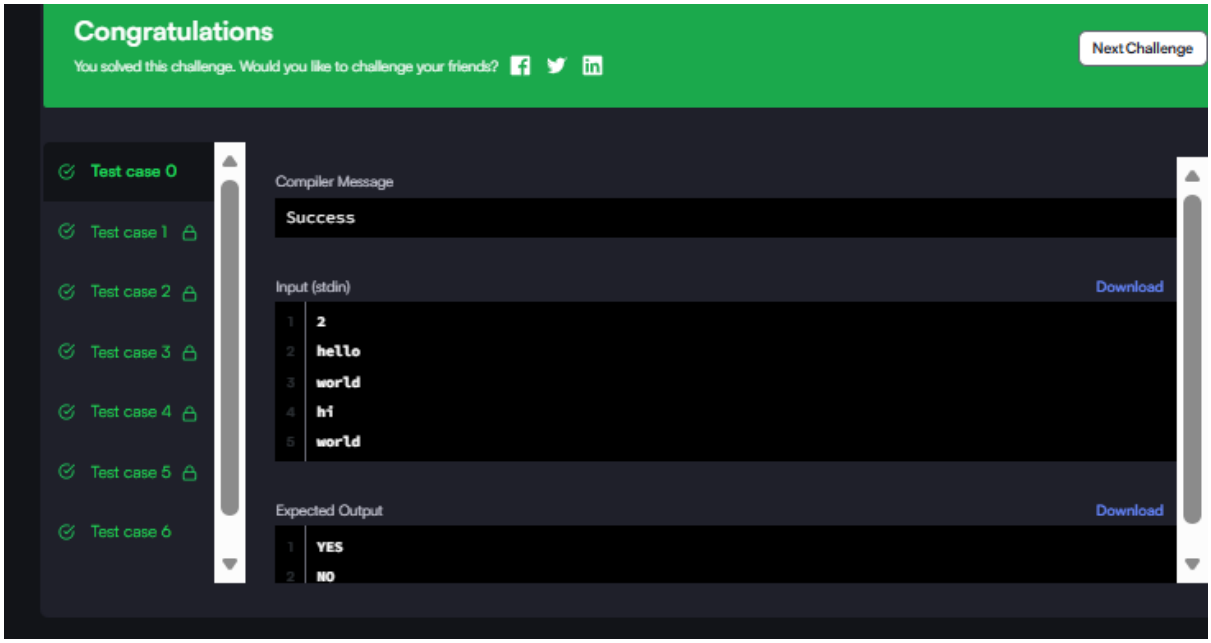
1. Read the two strings.
2. Convert one string into a set of characters.
3. Iterate through the other string:
4. If any character exists in the set, return "YES".
5. If no common character is found, return "NO".

Program:-

```
import java.io.*;

import java.util.*;

class Result {

   public static String twoStrings(String s1, String s2) {

      Set<Character> set = new HashSet<>();

      for (char c : s1.toCharArray()) set.add(c);

      for (char c : s2.toCharArray()) {

         if (set.contains(c)) return "YES";

      }

      return "NO";

   }

}

public class Solution {

   public static void main(String[] args) throws IOException {

      BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
```

```
        int t = Integer.parseInt(br.readLine().trim());
        for (int i = 0; i < t; i++) {
            String s1 = br.readLine().trim();
            String s2 = br.readLine().trim();
            System.out.println(Result.twoStrings(s1, s2));
        }
    }
}
```

Output:-



Result:-Thus,To determine if two given strings share a common substring is are successfully executed.

Task4:

Rotate String

Aim:- To check if one string s can be rotated (by repeatedly moving the first character to the end) to become another string goal.

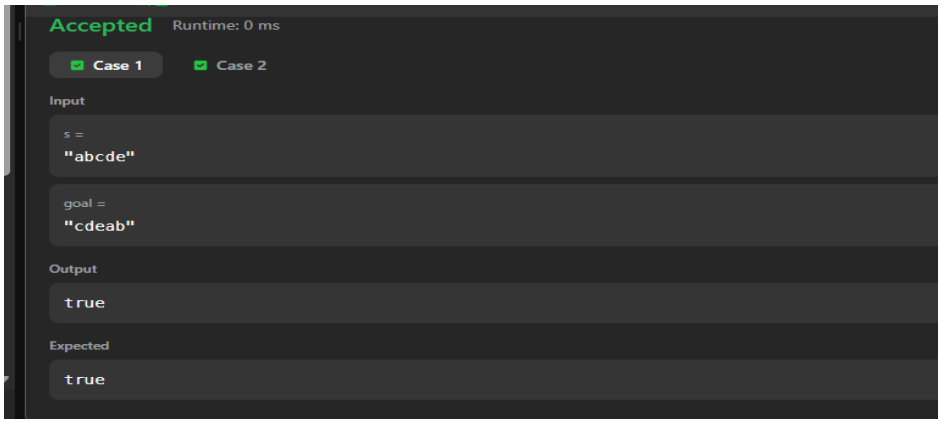
Algorithm:-

1. If the lengths of s and goal differ → return false.
2. Concatenate s with itself → s + s.
3. If goal is a substring of this doubled string → return true.
4. Otherwise → return false.

Program:-

```
class Solution {
   public boolean rotateString(String s, String goal) {
      if (s.length() != goal.length()) return false;
      String doubled = s + s;
      return doubled.contains(goal);
   }
}
```

Output:-



Result:-Thus,To check if one string s can be rotated (by repeatedly moving the first character to the end) to become another string goal is are successfully executed.

Task5:

Mars Exploration

Aim:-To determine how many characters in the received signal differ from the expected repeated "SOS" pattern. Each altered character represents radiation interference.

Algorithm:-

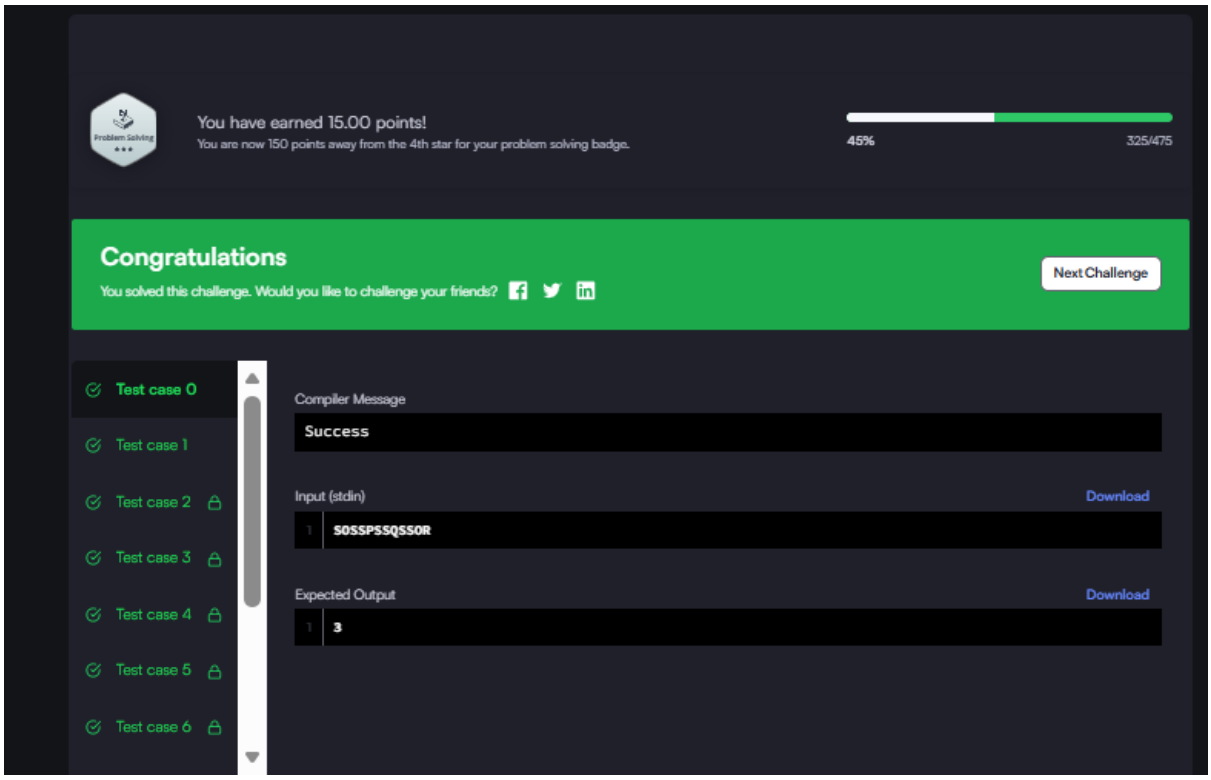
1. The original message is always "SOS" repeated $\frac{\text{length of received string}}{3}$ times.
2. Iterate through the received string:
3. Compare each character with the expected "SOS" character at that position (i % 3).
4. If they differ, increment a counter.
5. Return the counter as the number of altered characters.

Program:-

```
import java.io.*;

class Result {

   public static int marsExploration(String s) {

      int count = 0;

      String pattern = "SOS";

      for (int i = 0; i < s.length(); i++) {

         if (s.charAt(i) != pattern.charAt(i % 3)) {

            count++;

         }

      }

      return count;

   }

}
```

```
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        String s = br.readLine().trim();
        System.out.println(Result.marsExploration(s));
    }
}
```

Output:-



Result:-Thus, To determine how many characters in the received signal differ from the expected repeated "SOS" pattern. Each altered character represents radiation interference is are successfully executed.

Task6:

Find All Anagrams in a String

Aim:-Given two strings s and p, return all starting indices in s where the substring is an anagram of p.

Algorithm:-

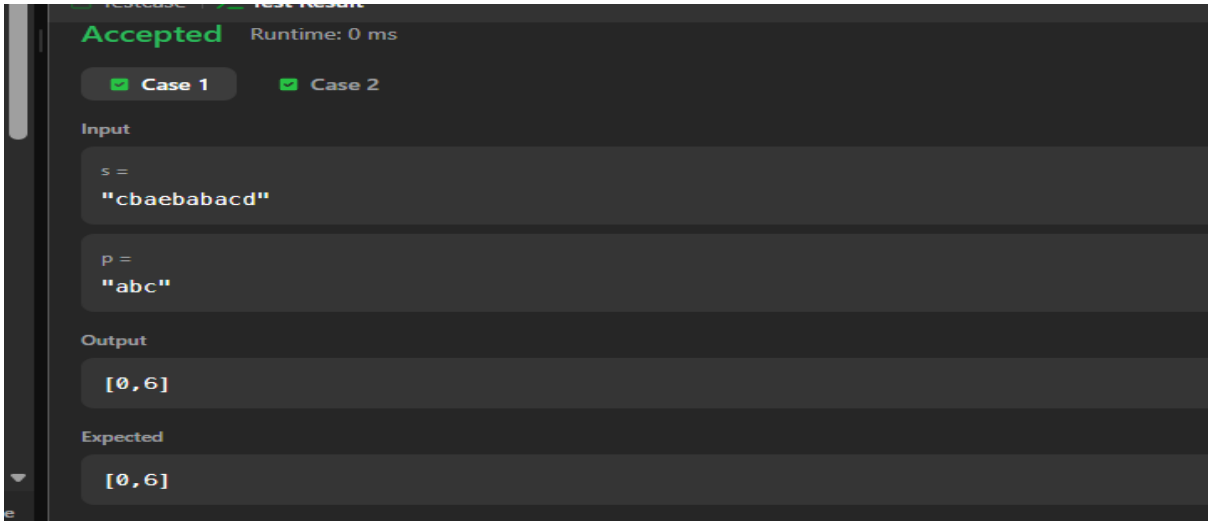
1. Count the frequency of characters in p.
2. Use a sliding window of size p.length() over s.
3. Maintain a frequency count for the current window.
4. If the window's frequency matches p's frequency → record the start index.
5. Slide the window forward by one character:
6. Remove the leftmost character from the count.
7. Add the new rightmost character.
8. Continue until the end of s.

Program:-

```
import java.util.*;

class Solution {

   public List<Integer> findAnagrams(String s, String p) {

      List<Integer> result = new ArrayList<>();

      if (s.length() < p.length()) return result;

      int[] pCount = new int[26];

      int[] sCount = new int[26];

      for (char c : p.toCharArray()) pCount[c - 'a']++;

      int window = p.length();

      for (int i = 0; i < s.length(); i++) {

         sCount[s.charAt(i) - 'a']++;

         if (i >= window) {

            sCount[s.charAt(i - window) - 'a']--;
```

```
            }
            if (Arrays.equals(pCount, sCount)) {
                result.add(i - window + 1);
            }
        }
        return result;
    }
}
```

Output:-



Result:-Thus, Given two strings s and p, return all starting indices in s where the substring is an anagram of p is are successfully executed.

Task:7

Find the Index of the First Occurrence in a String

Aim:-To return the index of the first occurrence of needle in haystack. If needle is not found, return -1.

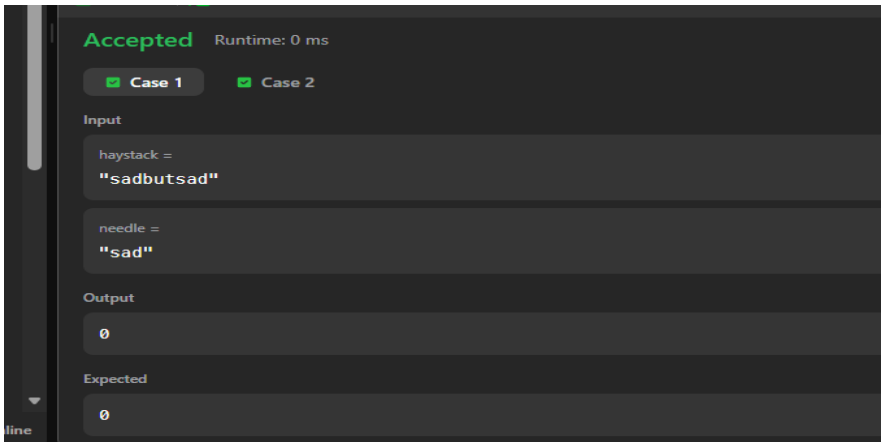
Algorithm:-

1. If needle is longer than haystack, return -1.
2. Use built-in string search (indexOf) or implement a manual check:
3. Iterate through haystack up to haystack.length - needle.length.
4. Compare substrings of length needle.length with needle.
5. If a match is found, return the index.
6. If no match is found, return -1.

Program:-

```
class Solution {
   public int strStr(String haystack, String needle) {
      return haystack.indexOf(needle);
   }
}
```

Output:-



Result:-Thus,To return the index of the first occurrence of needle in haystack. If needle is not found, return -1 is are successfully executed.

Task8:

Find the Index of the First Occurrence in a String

Aim:-To return the index of the first occurrence of needle in haystack. If needle is not found, return -1.

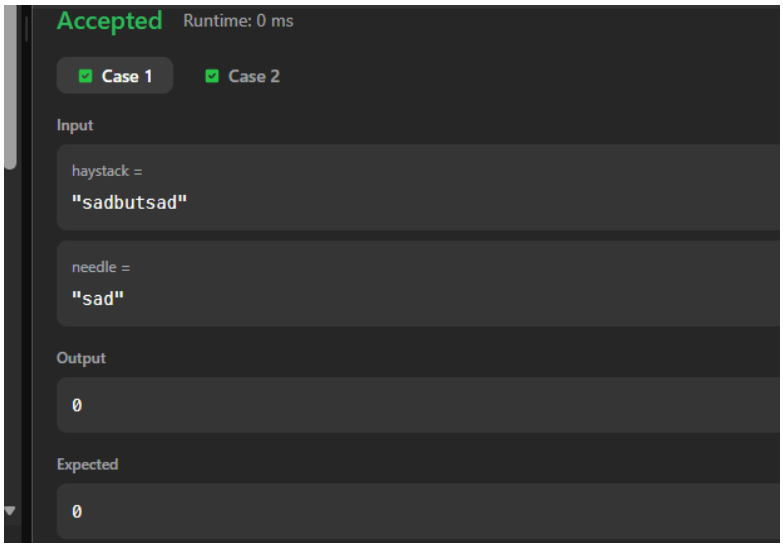
Algorithm:-

1. If needle is longer than haystack, return -1.
2. Use Java's built-in indexOf method, which efficiently searches for substrings.
3. Return the result directly.

Program:-

```
class Solution {
   public int strStr(String haystack, String needle) {
      return haystack.indexOf(needle);
   }
}
```

Output:-



Result:-Thus,To return the index of the first occurrence of needle in haystack. If needle is not found, return -1 is are successfully executed.

Task9:

Longest Palindromic Substring

Aim:- To find and return the longest substring of a given string s that is a palindrome (reads the same forward and backward).

Algorithm:-

1. A palindrome can be centered at:
2. A single character (odd-length palindrome).
3. Between two characters (even-length palindrome).
4. For each index in the string:
5. Expand outward while the left and right characters match.
6. Track the longest palindrome found.
7. Return the longest substring.

Program:-

```
class Solution {
   public String longestPalindrome(String s) {
      if (s == null || s.length() < 1) return "";
      int start = 0, end = 0;
      for (int i = 0; i < s.length(); i++) {
         int len1 = expandFromCenter(s, i, i);       // odd length
         int len2 = expandFromCenter(s, i, i + 1);   // even length
         int len = Math.max(len1, len2);
         if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
         }
```

```
        }
        return s.substring(start, end + 1);
    }

    private int expandFromCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) ==
s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1;
    }
}
```

Output:-

Result:-Thus, To find and return the longest substring of a given string s that is a palindrome (reads the same forward and backward) is are successfully executed.

Task10:

Circular Palindromes

Aim:-The goal of this program is to solve the **Circular Palindromes** challenge:For each rotation of a string S, find the maximum length of any palindromic substring fully contained in that rotation.

Algorithm:-

1. **Input Handling**
   - Reads n(length of string) and the string s.
   - Creates a doubled string s2 of length $2n$to represent all rotations.
2. **Palindrome Preprocessing**
   - Uses a custom palindrome radius function (palindrome) similar to Manacher's algorithm to compute the maximum palindrome length centered at each position in s2.
3. **Event Scheduling**
   - Builds an array of "events" (es) that represent when palindromes start, become active, and expire as the rotation window slides.
   - Each event is encoded as a long integer with position and index.
4. **Max Heaps**
   - Maintains three heaps (inc, dec, flat) to track palindromes that are growing, shrinking, or flat relative to the rotation window.
   - As the rotation window moves, events are processed and palindromes are added/removed from heaps.
5. **Output**
   - For each rotation, computes the maximum palindrome length using the heaps and prints it.

Program:-

```
import java.io.ByteArrayInputStream;

import java.io.IOException;

import java.io.InputStream;
```

```
import java.io.PrintWriter;
import java.util.Arrays;
import java.util.InputMismatchException;

public class E2 {
    InputStream is;
    PrintWriter out;
    String INPUT = "";

    void solve()
    {
        int n = ni();
        char[] s = ns(n);
        char[] s2 = new char[2*n];
        for(int i = 0;i < n;i++){
            s2[i] = s2[i+n] = s[i];
        }
        int[] pal = palindrome(s2);
//        tr(pal, pal.length, n);
        long[] es = new long[16*n];
        int p = 0;
        for(int i = 0;i < 4*n;i+=2){
            pal[i] = Math.min(pal[i], n-((n&1)^1));
            es[p++] = (long)(i/2)<<32|i;
            es[p++] = (long)(i/2+pal[i]/2)<<32|i;
            es[p++] = (long)(i/2+n-pal[i]/2-1)<<32|i;
            es[p++] = (long)(i/2+n)<<32|i;
```

```
}
for(int i = 1;i < 4*n;i+=2){
    pal[i] = Math.min(pal[i], n-((n&1)));
    es[p++] = (long)(i/2)<<32|i;
    es[p++] = (long)(i/2+pal[i]/2)<<32|i;
    es[p++] = (long)(i/2+n-pal[i]/2)<<32|i;
    es[p++] = (long)(i/2+n)<<32|i;
}

Arrays.sort(es, 0, p);
MaxHeap inc = new MaxHeap(4*n+1);
MaxHeap dec = new MaxHeap(4*n+1);
MaxHeap flat = new MaxHeap(4*n+1);

int[] st = new int[4*n];
int q = 0;
for(int i = 0;i < 2*n-1;i++){
    while(q < p && es[q]>>>32 <= i){
        int ind = (int)es[q];
        if(st[ind] == 0){
            inc.add(ind, (pal[ind]&1)-2*i);
        }else if(st[ind] == 1){
            inc.remove(ind);
            flat.add(ind, pal[ind]);
        }else if(st[ind] == 2){
            flat.remove(ind);
            dec.add(ind, pal[ind]+2*i);
```

```
            }else if(st[ind] == 3){
                dec.remove(ind);
            }
            st[ind]++;
            q++;
        }
        if(i >= n-1){
//              tr("i", i);
            int max = 0;
            if(inc.size() > 0)max = Math.max(inc.max()+2*i, max);
//              tr(max);
            if(dec.size() > 0)max = Math.max(dec.max()-2*i, max);
//              tr(max);
            max = Math.max(flat.max(), max);
//              tr(max);
            out.println(max);
        }
    }
}
public static class MaxHeap {
    public int[] a;
    public int[] map;
    public int[] imap;
    public int n;
    public int pos;
    public static int INF = Integer.MIN_VALUE;
```

```
public MaxHeap(int m)
{
    n = m+2;
    a = new int[n];
    map = new int[n];
    imap = new int[n];
    Arrays.fill(a, INF);
    Arrays.fill(map, -1);
    Arrays.fill(imap, -1);
    pos = 1;
}

public int add(int ind, int x)
{
    int ret = imap[ind];
    if(imap[ind] < 0){
        a[pos] = x; map[pos] = ind; imap[ind] = pos;
        pos++;
        up(pos-1);
    }
    return ret != -1 ? a[ret] : x;
}

public int update(int ind, int x)
{
    int ret = imap[ind];
    if(imap[ind] < 0){
```

```
            a[pos] = x; map[pos] = ind; imap[ind] = pos;
            pos++;
            up(pos-1);
        }else{
            int o = a[ret];
            a[ret] = x;
            up(ret);
            down(ret);
//              if(a[ret] < o){
//                  up(ret);
//              }else{
//                  down(ret);
//              }
        }
        return x;
    }

    public int remove(int ind)
    {
        if(pos == 1)return INF;
        if(imap[ind] == -1)return INF;

        pos--;
        int rem = imap[ind];
        int ret = a[rem];
        map[rem] = map[pos];
        imap[map[pos]] = rem;
```

```
        imap[ind] = -1;
        a[rem] = a[pos];
        a[pos] = INF;
        map[pos] = -1;

        up(rem);
        down(rem);
        return ret;
    }

    public int max() { return a[1]; }
    public int argmax() { return map[1]; }
    public int size() {    return pos-1; }

    private void up(int cur)
    {
        for(int c = cur, p = c>>>1;p >= 1 && a[p] < a[c];c>>>=1, p>>>=1){
            int d = a[p]; a[p] = a[c]; a[c] = d;
            int e = imap[map[p]]; imap[map[p]] = imap[map[c]]; imap[map[c]] =
e;
            e = map[p]; map[p] = map[c]; map[c] = e;
        }
    }

    private void down(int cur)
    {
        for(int c = cur;2*c < pos;){
            int b = a[2*c] > a[2*c+1] ? 2*c : 2*c+1;
```

```
            if(a[b] > a[c]){
                int d = a[c]; a[c] = a[b]; a[b] = d;
                int e = imap[map[c]]; imap[map[c]] = imap[map[b]]; imap[map[b]]
= e;
                e = map[c]; map[c] = map[b]; map[b] = e;
                c = b;
            }else{
                break;
            }
        }
    }
}

public static int[] palindrome(char[] str)
{
    int n = str.length;
    int[] r = new int[2*n];
    int k = 0;
    for(int i = 0, j = 0;i < 2*n;i += k, j = Math.max(j-k, 0)){
        // normally
        while(i-j >= 0 && i+j+1 < 2*n && str[(i-j)/2] == str[(i+j+1)/2])j++;
        r[i] = j;

        // skip based on the theorem
        for(k = 1;i-k >= 0 && r[i]-k >= 0 && r[i-k] != r[i]-k;k++){
            r[i+k] = Math.min(r[i-k], r[i]-k);
        }
    }
```

```
        return r;
    }



    void run() throws Exception
    {
        is = INPUT.isEmpty() ? System.in : new
ByteArrayInputStream(INPUT.getBytes());
        out = new PrintWriter(System.out);


        long s = System.currentTimeMillis();
        solve();
        out.flush();
        if(!INPUT.isEmpty())tr(System.currentTimeMillis()-s+"ms");
    }

    public static void main(String[] args) throws Exception { new E2().run(); }

    private byte[] inbuf = new byte[1024];
    private int lenbuf = 0, ptrbuf = 0;

    private int readByte()
    {
        if(lenbuf == -1)throw new InputMismatchException();
        if(ptrbuf >= lenbuf){
            ptrbuf = 0;
            try { lenbuf = is.read(inbuf); } catch (IOException e) { throw new
InputMismatchException(); }
```

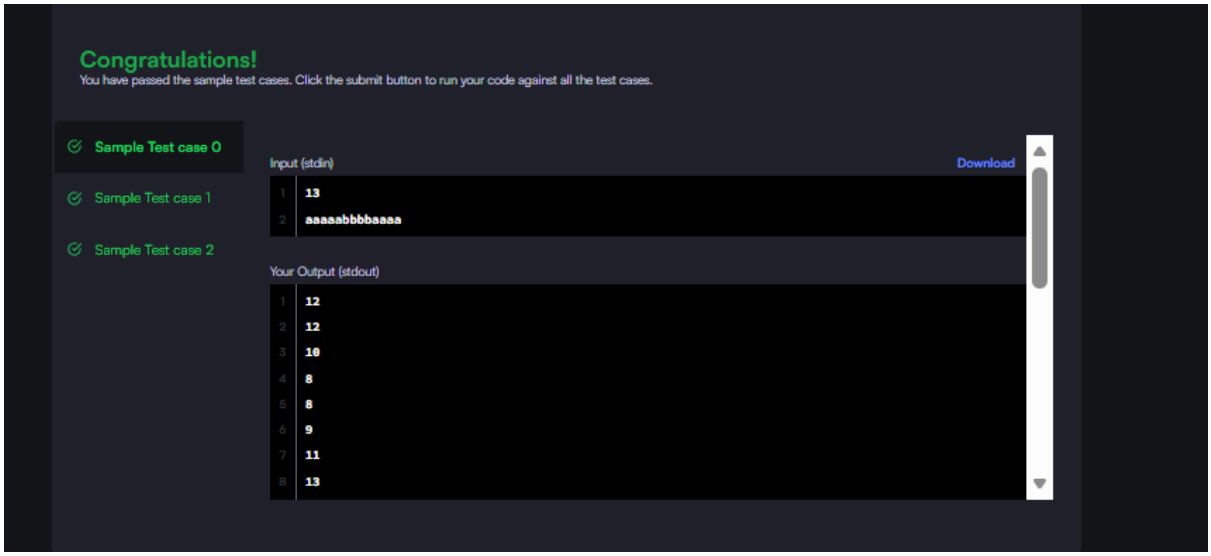
```
        if(lenbuf <= 0)return -1;
    }
    return inbuf[ptrbuf++];
}

private boolean isSpaceChar(int c) { return !(c >= 33 && c <= 126); }
private int skip() { int b; while((b = readByte()) != -1 && isSpaceChar(b));
return b; }

private double nd() { return Double.parseDouble(ns()); }
private char nc() { return (char)skip(); }

private String ns()
{
    int b = skip();
    StringBuilder sb = new StringBuilder();
    while(!(isSpaceChar(b))){
        sb.appendCodePoint(b);
        b = readByte();
    }
    return sb.toString();
}

private char[] ns(int n)
{
    char[] buf = new char[n];
    int b = skip(), p = 0;
    while(p < n && !(isSpaceChar(b))){
```

```
        buf[p++] = (char)b;
        b = readByte();
    }
    return n == p ? buf : Arrays.copyOf(buf, p);
}

private char[][] nm(int n, int m)
{
    char[][] map = new char[n][];
    for(int i = 0;i < n;i++)map[i] = ns(m);
    return map;
}

private int[] na(int n)
{
    int[] a = new int[n];
    for(int i = 0;i < n;i++)a[i] = ni();
    return a;
}

private int ni()
{
    int num = 0, b;
    boolean minus = false;
    while((b = readByte()) != -1 && !((b >= '0' && b <= '9') || b == '-'));
    if(b == '-'){
        minus = true;
```

```
        b = readByte();
    }

    while(true){
        if(b >= '0' && b <= '9'){
            num = num * 10 + (b - '0');
        }else{
            return minus ? -num : num;
        }
        b = readByte();
    }
}

private long nl()
{
    long num = 0;
    int b;
    boolean minus = false;
    while((b = readByte()) != -1 && !((b >= '0' && b <= '9') || b == '-'));
    if(b == '-'){
        minus = true;
        b = readByte();
    }

    while(true){
        if(b >= '0' && b <= '9'){
            num = num * 10 + (b - '0');
```

```
        }else{
            return minus ? -num : num;
        }
        b = readByte();
    }
}

  private static void tr(Object... o) {
System.out.println(Arrays.deepToString(o)); }
}
```

Output:-



Result:-Thus,To the goal of this program is to solve the **Circular Palindromes** challenge:For each rotation of a string *S*, find the maximum length of any palindromic substring fully contained in that rotation is are successfully executed.